

Sprawozdanie 3

Eksploracja danych

Emilia Porczyńska, Michał Szostak

2026-06-04

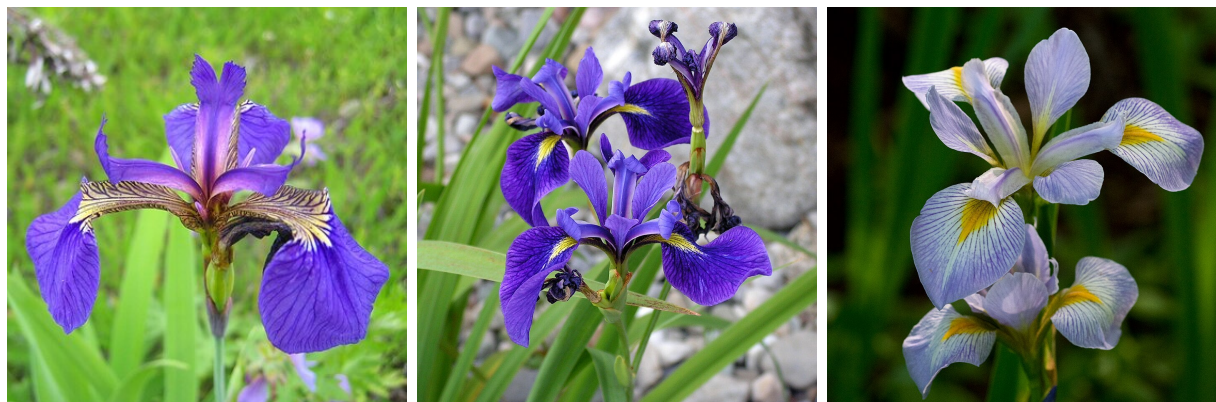
Spis treści

1	Część 1 – klasyfikacja	2
1.1	Wstęp	2
1.2	Przypomnienie struktury danych	2
1.3	Podział danych na zbiór uczący i testowy	3
1.4	Konstrukcja klasyfikatora i wyznaczanie prognoz	3
1.5	Ocena klasyfikacji	4
1.6	Model liniowy dla rozszerzonej przestrzeni klas	6
2	Część 2 – porównanie metod klasyfikacji	7
2.1	Wstęp	7
2.2	Opis i wstępna analiza danych	8
2.3	Klasyfikacja i ocena dokładności	13
2.4	Porównanie metod	16
2.5	Różne parametry oraz podzbiory cech	17
2.6	Podsumowanie	20

1 Część 1 – klasyfikacja

1.1 Wstęp

Pracujemy na tym samym zbiorze danych co w poprzednim sprawozdaniu. Zbiór zawiera wyniki pomiarów przedstawicieli trzech gatunków irysów: *Iris setosa*, *Iris versicolor* oraz *Iris virginica*. Pomiary dotyczą długości oraz szerokości poszczególnych części kwiatu – działki kielicha (ang. *sepal*) oraz płatków (ang. *petal*). W przeciągu ostatniego miesiąca wygląd kwiatów się nie zmienił i nadal wyglądają one tak:



Rysunek 1: *Iris setosa* (źródło), *Iris versicolor* (źródło) oraz *Iris virginica* (źródło)

1.2 Przypomnienie struktury danych

W naszych danych znajdują się pomiary 150 irysów, po 50 z każdego gatunku. Dla każdego z nich mamy wyniki czterech pomiarów, oraz informację o tym do którego gatunku należy:

	Typ	Opis
Sepal.Length	numeric	długość działki kielicha (w cm)
Sepal.Width	numeric	szerokość działki kielicha (w cm)
Petal.Length	numeric	długość płatka (w cm)
Petal.Width	numeric	szerokość płatka (w cm)
Species	factor	gatunek irysa (<i>setosa</i> / <i>versicolor</i> / <i>virginica</i>)

Tabela 1: Typy oraz opisy cech

Wartości pomiarów prezentują się następująco:

Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
Min.: 4.300	Min.: 2.000	Min.: 1.000	Min.: 0.100
1st Qu.: 5.100	1st Qu.: 2.800	1st Qu.: 1.600	1st Qu.: 0.300
Median: 5.800	Median: 3.000	Median: 4.350	Median: 1.300
Mean: 5.843	Mean: 3.057	Mean: 3.758	Mean: 1.199
3rd Qu.: 6.400	3rd Qu.: 3.300	3rd Qu.: 5.100	3rd Qu.: 1.800
Max.: 7.900	Max.: 4.400	Max.: 6.900	Max.: 2.500

Tabela 2: Podsumowanie danych

1.3 Podział danych na zbiór uczący i testowy

W celu oceny jakości działania klasyfikatora dzielimy zbiór danych na zbiór uczący `train_set` i zbiór testowy `test_set` w proporcjach 2 do 1:

```
set.seed(67) # ustawiamy seed losowania

n <- nrow(dane) # liczba obserwacji

# losujemy które obserwacje będą w zbiorze uczącym a które w testowym
train_index <- sample(1:n, size = round(2/3 * n))

# tworzymy osobne ramki danych dla obu zbiorów
train_set <- dane[train_index, ]
test_set <- dane[-train_index, ]
```

Zbiór uczący zawiera 100 obserwacji, a zbiór testowy 50.

1.4 Konstrukcja klasyfikatora i wyznaczanie prognoz

Będziemy konstruować klasyfikator oparty na regresji liniowej. W tym celu na podstawie zbioru uczącego (`train_set`) będziemy estymować parametry modelu.

Aby to zrobić, musimy najpierw zakodować klasy przy użyciu zmiennych binarnych (indykatory klas):

```
etykietki.klas <- train_set$Species

K <- length(unique(etykietki.klas)) # liczba klas (gatunków)
n.train <- nrow(train_set)          # liczba obserwacji uczących

# tworzymy macierz wskaźników klas i przypisujemy jej odpowiednie wartości
Y.train <- matrix(0, nrow = n.train, ncol = K)
etykietki.num <- as.numeric(etykietki.klas)
```

```
for(k in 1:K)
  Y.train[etykietki.num == k, k] <- 1
```

Następnie estymujemy parametry modelu za pomocą zbioru uczącego:

```
# macierz cech z dodatkową kolumną jedynek (wyraz wolny)
X.train <- as.matrix(cbind(1, train_set[,1:4]))

# estymacja współczynników metodą najmniejszych kwadratów:  $B=(X'X)^{-1} X'Y$ 
B.hat <- solve(t(X.train) %*% X.train) %*%
          t(X.train) %*% Y.train
```

Dla każdej obserwacji – zarówno ze zbioru uczącego i testowego – liczymy estymowane indykatory klas:

```
# dopasowane wskaźniki klas dla zbioru uczącego
Y.hat.train <- X.train %*% B.hat

# klasa = kolumna z największą wartością
pred.train <- apply(Y.hat.train, 1, which.max)

# to samo dla zbioru testowego (kolumna jedynek jak w zbiorze uczącym)
X.test <- cbind(1, test_set[,1:4])
X.test <- as.matrix(X.test)

Y.hat.test <- X.test %*% B.hat
pred.test <- apply(Y.hat.test, 1, which.max)
```

1.5 Ocena klasyfikacji

W celu oceny klasyfikacji wyliczamy macierz pomyłek (ang. *confusion matrix*) oraz błąd klasyfikacji na zbiorze uczącym oraz testowym.

	<i>setosa</i>	<i>versicolor</i>	<i>virginica</i>
<i>setosa</i>	28	0	0
<i>versicolor</i>	0	28	9
<i>virginica</i>	0	6	29

Tabela 3: Macierz pomyłek dla zbioru uczącego (model liniowy). Wiersz oznacza faktyczny gatunek, a kolumna estymowany.

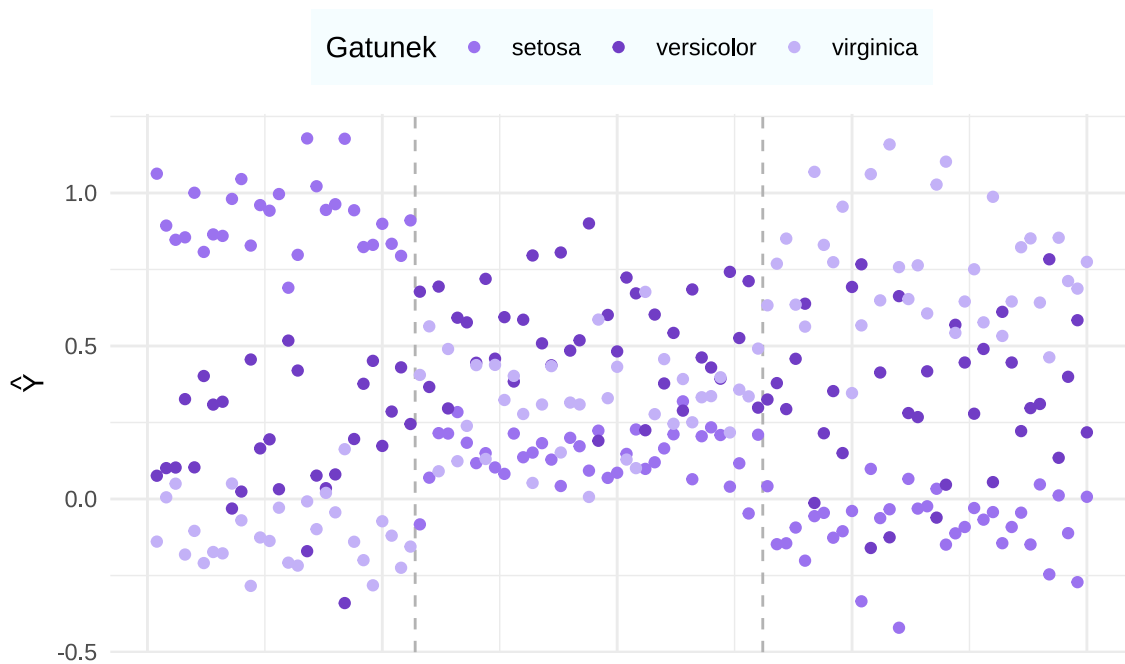
Na powyższej tabeli widzimy wyniki dopasowania gatunków do kwiatów. Błąd klasyfikacji (proporcja źle sklasyfikowanych obserwacji do wszystkich obserwacji) wynosi 15%. Taka wysoka wartość wynika z mylenia przez model gatunków *versicolor* oraz *virginica* – jak na to że jest to zbiór uczący, sugeruje to że ciężko jest te gatunki rozdzielić.

Zjawisko maskowania klas nie występuje – gdyby występowało, któraś z klas byłaby przewidywana znacząco rzadziej niż występuje. W naszym przypadku tak nie jest. Pomaga nam to, że liczba $p = 4$ zmiennych objaśniających jest większa od liczby $K = 3$ klas, dzięki czemu „wymiarowość” gra trochę na naszą korzyść.

	<i>setosa</i>	<i>versicolor</i>	<i>virginica</i>
<i>setosa</i>	22	0	0
<i>versicolor</i>	0	11	2
<i>virginica</i>	0	5	10

Tabela 4: Macierz pomyłek dla zbioru testowego (model liniowy). Wiersz oznacza faktyczny gatunek, a kolumna estymowany.

Dla zbioru testowego błąd klasyfikacji wynosi 14%; zbliżona wartość błędów dla zbioru uczącego i testowego świadczy, że model nie jest przetrenowany i dobrze klasyfikuje dane.



Rysunek 2: Wartości wskaźników klas $\hat{Y}$ (estymatorów indykatorów) dla modelu liniowego (zbiór uczący). Oś x nic nie oznacza i ma na celu tylko rozdzielenie gatunków. Od lewej: *setosa*, *versicolor*, *virginica*.

Na wykresie widać, że najlepszą separowalność uzyskano dla kwiatów z gatunku *setosa*, o czym świadczą wysokie wartości wskaźników dla ich własnej klasy i niskie dla pozostałych.

W przypadku *versicolor* i *virginica*, wartości wskaźników właściwych klas są niższe niż w przypadku *setosa*, niewiele większe od wartości wskaźników o drugich najwyższych wartościach – wskaźników *virginica* w przypadku *versicolor* oraz vice versa. Powoduje to trudność w

rozdzieleniu przez model kwiatów tych dwóch gatunków, co zostało zauważone już przy macierzy pomyłek.

1.6 Model liniowy dla rozszerzonej przestrzeni klas

Teraz przeprowadzimy klasyfikację gatunków irysów na rozszerzonej przestrzeni cech. W tym celu zbudujemy nowy model liniowy dla tych cech. Nasza nowa przestrzeń cech zawiera oryginalne zmienne PL, PW, SL i SW oraz składniki wielomianowe stopnia 2: $PL^2, PW^2, SL^2, SW^2, PL \cdot PW, PL \cdot SL, PL \cdot SW, PW \cdot SL, PW \cdot SW$ oraz $SL \cdot SW$.

Model konstruujemy analogicznie jak poprzednio z uprzednim dodaniem nowych cech.

W celu porównania dokładności klasyfikacji modelu z poprzednim, przeanalizujemy wyniki klasyfikacji w podobny sposób jak wcześniejsze, zaczynając od analizy macierzy pomyłek oraz błędów klasyfikacji.

	setosa	versicolor	virginica
setosa	28	0	0
versicolor	0	36	1
virginica	0	1	34

Tabela 5: Macierz pomyłek dla zbioru uczącego (model wielomianowy)

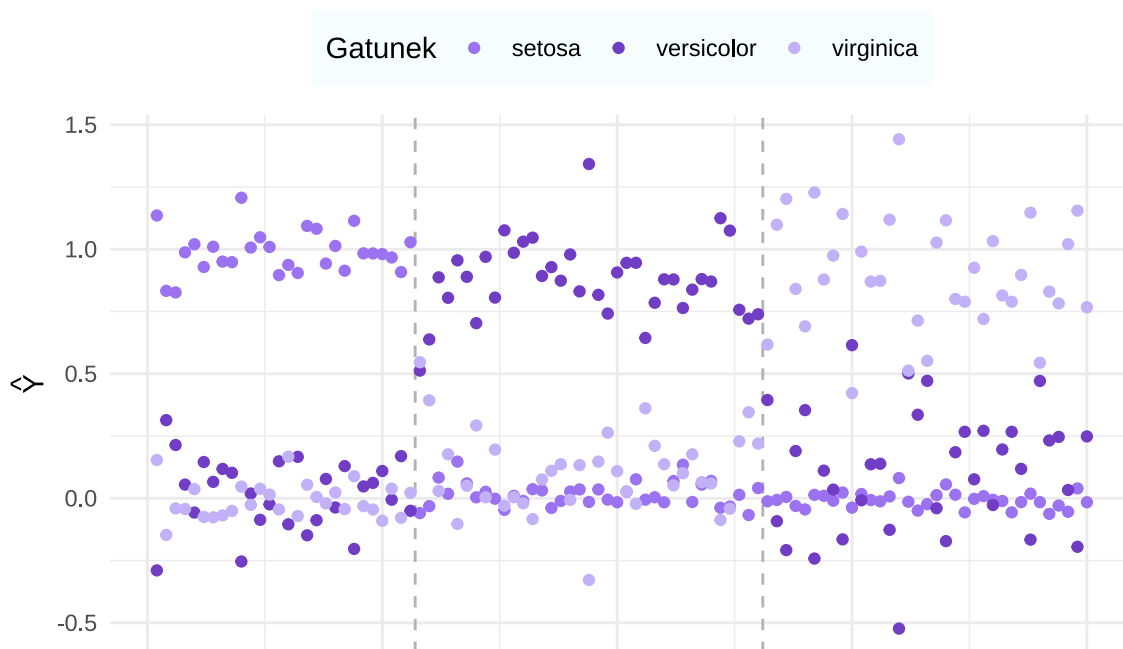
Od razu widzimy lepszą separowalność gatunków dla samego zbioru uczącego niż w przypadku poprzedniego modelu – błąd klasyfikacji wynosi jedynie 2% w porównaniu do wcześniejszych 15%. Sugeruje to, że zależności gatunków między irysami nie są liniowe, ponieważ wprowadzenie dodatkowych cech pozwoliło na lepsze odseparowanie klas.

Ponownie nie występuje zjawisko maskowania klas.

	setosa	versicolor	virginica
setosa	22	0	0
versicolor	0	13	0
virginica	0	3	12

Tabela 6: Macierz pomyłek dla zbioru testowego (model wielomianowy)

Na zbiorze testowym błąd klasyfikacji wynosi 6%. Jest to gorszy wynik niż w zbiorze uczącym. Może to sugerować drobne przeuczenie modelu, lecz może równie dobrze być to błąd statystyczny wynikający z małej liczebności próby. Wynik ten wciąż jest lepszy od wyniku w poprzednim modelu (14%).



Rysunek 3: Wartości wskaźników klas $\hat{Y}$ (estymatorów indyktorów) dla modelu wielomianowego (zbiór uczący). Oś x nie oznacza i ma na celu tylko rozdzielenie gatunków. Od lewej: *setosa*, *versicolor*, *virginica*.

W nowym modelu wykres wskaźników klas wygląda dużo przyjemniej. Dla wszystkich gatunków wartości właściwych wskaźników są prawie wszystkie w okolicy 1, a niewłaściwych w okolicy 0.

Wciąż jest odrobinę lepiej w przypadku *setosa*, niż w przypadku *versicolor* i *virginica*, lecz dwa ostatnie gatunki są o wiele lepiej traktowane niż w poprzednim modelu i tylko czasami są mylone.

Ogółem, model rozszerzony o składniki wielomianowe stopnia 2 radzi sobie lepiej z klasyfikacją niż model bez tych składników, co pozwala nam na lepszą klasyfikację gatunków naszych kwiatów i sprawia że jesteśmy szczęśliwsi.

2 Część 2 – porównanie metod klasyfikacji

2.1 Wstęp

W tej części raportu będziemy pracować na zbiorze danych Wine z R-pakietu `HDclassif`. Zbiór danych zawiera informacje o wynikach analizy chemicznej win uprawianych w tych samych regionach Włoch, lecz będących trzech różnych odmian.

W naszym zbiorze danych, ani na jego stronie nie ma informacji o tym jakie to są odmiany, jednak w *innym wariancie zbioru* są podane ich nazwy: *Barolo*, *Grignolino* oraz *Barbera*.

Naszym zadaniem będzie zastosowanie i porównanie algorytmów klasyfikacji k-najbliższych

sąsiadów, drzewa klasyfikacyjnego oraz naiwnego klasyfikatora bayesowskiego na tym zbiorze danych.

Cechą według której będziemy klasyfikować wina jest zmienna informująca o przynależności do odmiany wina.

2.2 Opis i wstępna analiza danych

Dane zawierają informacje o 178 winach. Dla każdego z nich mamy informacje o 14 cechach:

	Typ	Opis
class	integer	przynależność do klasy wina
V1	numeric	zawartość alkoholu
V2	numeric	kwas jabłkowy
V3	numeric	zawartość popiołu
V4	numeric	alkaliczność popiołu
V5	integer	zawartość magnezu
V6	numeric	łączna zawartość fenoli
V7	numeric	flawonoidy
V8	numeric	fenole nieflawonoidowe
V9	numeric	proantocyjaniny
V10	numeric	intensywność barwy
V11	numeric	odcień koloru
V12	numeric	OD280/OD315 rozcieńczonych win
V13	integer	zawartość proliny

Tabela 7: Typy oraz opisy cech zbioru wine

Cecha `class` będzie nas informować o przynależności do klasy, natomiast do klasyfikacji posłużą nam 13 pozostałych cech dotyczących różnych parametrów wina. Zmienna `class` zawiera informacje o przynależności do jednej z 3 klas, czyli odmian wina.

Porównując nasz zbiór ze wspomnianym wcześniej wariantem można się dowiedzieć, że klasa zakodowana u nas jako 1 to odmiana *Barolo*, klasa zakodowana jako 2 to *Grignolino*, a klasa zakodowana jako 3 to *Barbera*.

Zmieniamy odpowiednio etykiety w naszym zbiorze:

```
dane2$class <- factor(
  dane2$class,
  labels = c("Barolo", "Grignolino", "Barbera")
)
```

Alternatywny zbiór zawiera także opisowe nazwy zmiennych, jednak zachowujemy nazwy numeryczne aby nie utrudniać pracy na danych.

Zmienne V5 (zawartość magnezu) oraz V13 (zawartość proliny) zostały wczytane jako `integer`. Wynika to z tego, że w danych występują jedynie wartości całkowite, prawdopodobnie z powodu zaokrąglenia. Wartości tych cech jak najbardziej mogłyby być niecałkowite, więc zmieniamy ich typ na `numeric`.

W poniższej tabeli znajdują się statystyki opisowe zmiennych, dzięki którym możliwe będzie wstępne zapoznanie się z danymi oraz identyfikacja podstawowych nieprawidłowości.

class	V1	V2	V3	V4
Barolo :59	Min. :11.03	Min. :0.740	Min. :1.360	Min. :10.60
Grignolino:71	1st Qu.:12.36	1st Qu.:1.603	1st Qu.:2.210	1st Qu.:17.20
Barbera :48	Median :13.05	Median :1.865	Median :2.360	Median :19.50
	Mean :13.00	Mean :2.336	Mean :2.367	Mean :19.49
	3rd Qu.:13.68	3rd Qu.:3.083	3rd Qu.:2.558	3rd Qu.:21.50
	Max. :14.83	Max. :5.800	Max. :3.230	Max. :30.00

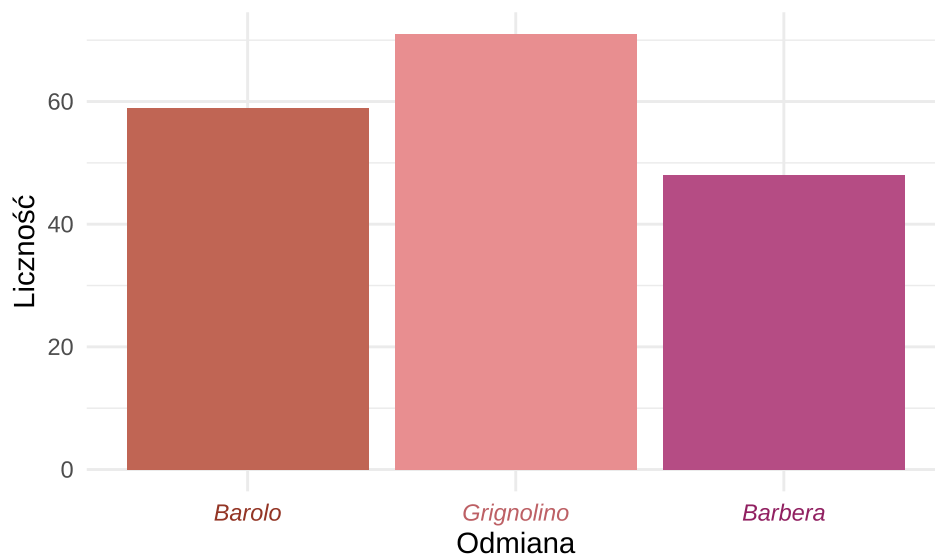
V5	V6	V7	V8	V9
Min. : 70.00	Min. :0.980	Min. :0.340	Min. :0.1300	Min. :0.410
1st Qu.: 88.00	1st Qu.:1.742	1st Qu.:1.205	1st Qu.:0.2700	1st Qu.:1.250
Median : 98.00	Median :2.355	Median :2.135	Median :0.3400	Median :1.555
Mean : 99.74	Mean :2.295	Mean :2.029	Mean :0.3619	Mean :1.591
3rd Qu.:107.00	3rd Qu.:2.800	3rd Qu.:2.875	3rd Qu.:0.4375	3rd Qu.:1.950
Max. :162.00	Max. :3.880	Max. :5.080	Max. :0.6600	Max. :3.580

V10	V11	V12	V13
Min. : 1.280	Min. :0.4800	Min. :1.270	Min. : 278.0
1st Qu.: 3.220	1st Qu.:0.7825	1st Qu.:1.938	1st Qu.: 500.5
Median : 4.690	Median :0.9650	Median :2.780	Median : 673.5
Mean : 5.058	Mean :0.9574	Mean :2.612	Mean : 746.9
3rd Qu.: 6.200	3rd Qu.:1.1200	3rd Qu.:3.170	3rd Qu.: 985.0
Max. :13.000	Max. :1.7100	Max. :4.000	Max. :1680.0

Tabela 8: Podsumowanie danych zbioru wine

W danych nie występują wartości NA. Braki zakodowane w inny sposób raczej nie występują – na pewno nie występują braki zapisane jako 0, ponieważ wartości 0 nie występują wcale.

W podsumowaniu możemy zauważyć także znacząco różną zmienność w przypadku zmiennej V13 (zawartość proliny) niż w przypadku pozostałych zmiennych. Może to wskazywać na ewentualną potrzebę standaryzacji przy niektórych metodach.

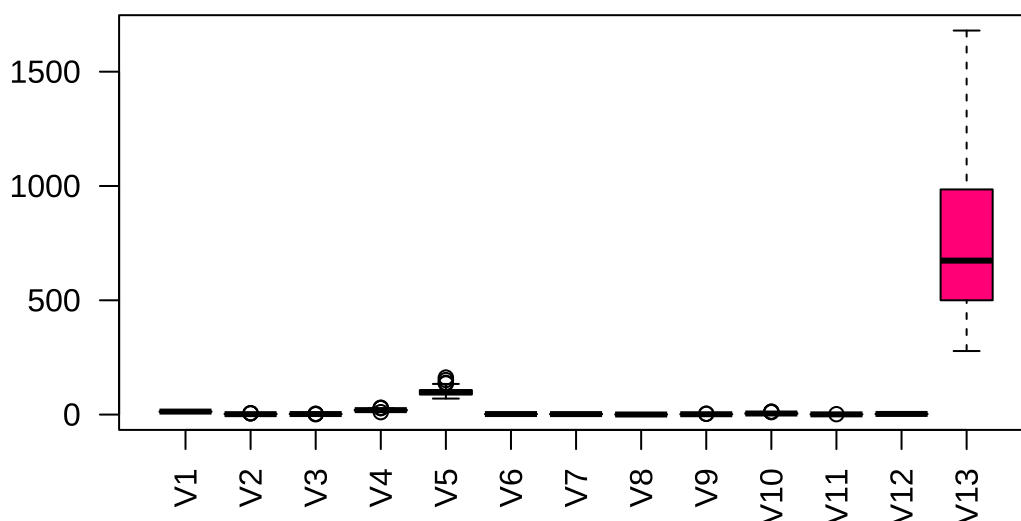


Rysunek 4: Rozkład licznosci odmian w zbiorze `wine`. Kolory słupków odpowiadają odmianom: *Barolo*, *Grignolino*, *Barbera*.

Na powyższym wykresie widzimy rozkład licznosci odmian. Możemy zauważyć, że najwięcej obserwacji należy do odmiany *Grignolino*, natomiast najmniej do odmiany *Barbera*. Różnice nie są jednak znaczące, licznosci klas są w miarę zbalansowane.

Klasyfikator przypisujący wszystkie obserwacje do najczęściej występującej klasy miałby błąd klasyfikacji na poziomie 60%, czyli częściej by się mylił niż zgadywał dobrze.

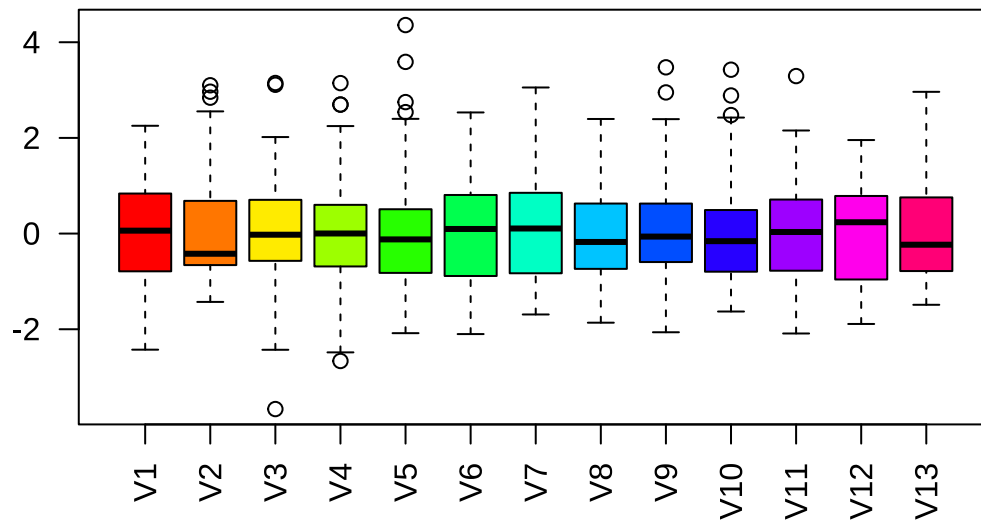
Poniższy wykres pokazuje rozrzut wartości dla wszystkich cech ilościowych:



Rysunek 5: Wykresy pudełkowe cech ilościowych przed standaryzacją.

Jak już zauważyliśmy wcześniej, zmienna `V13` (zawartość proliny) cechuje się znacznie wyższą zmiennością. W związku z tym konieczne jest zastosowanie standaryzacji aby algorytmy

klasyfikujące działały poprawnie.

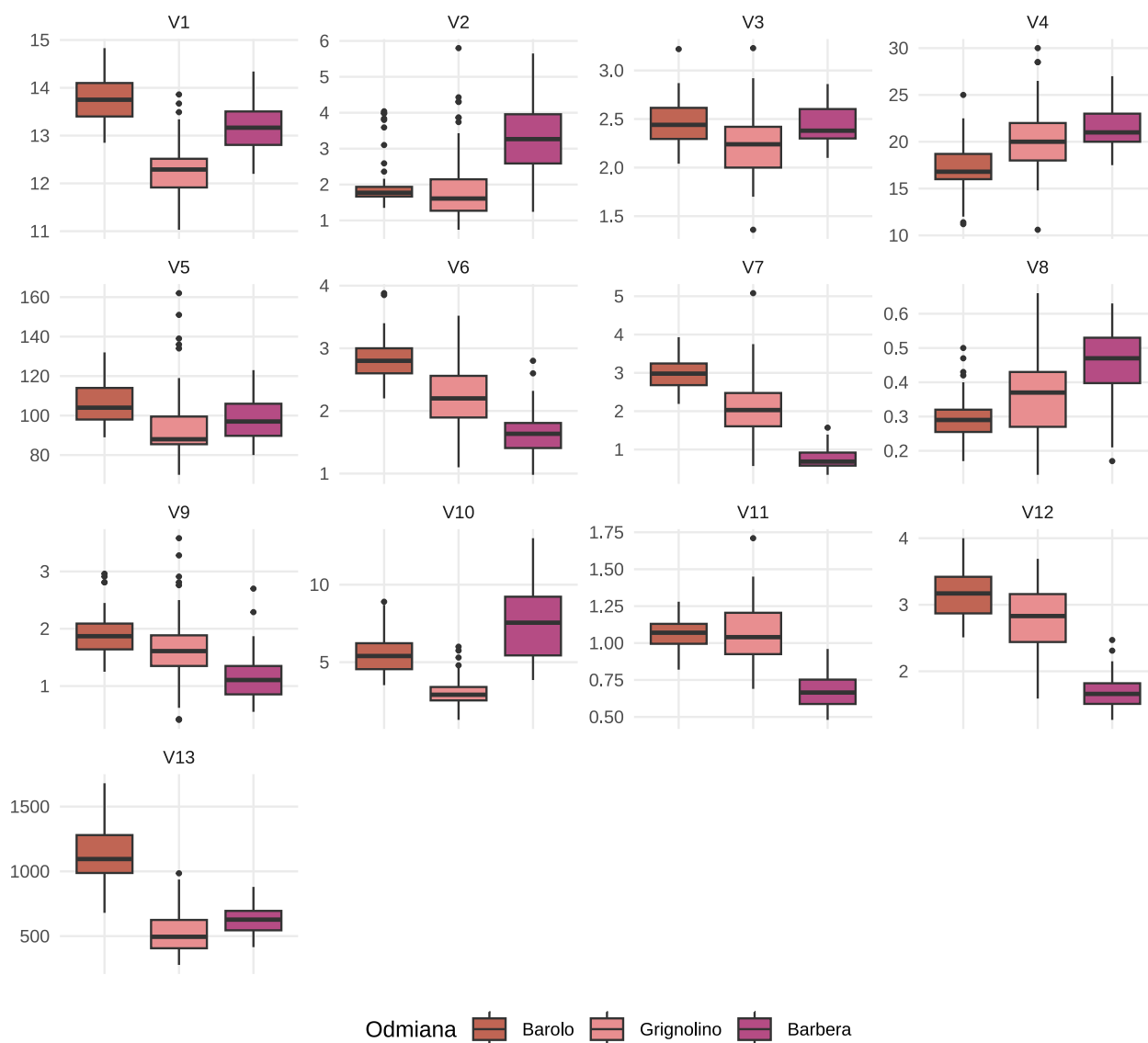


Rysunek 6: Wykresy pudełkowe cech ilościowych po standaryzacji.

Widzimy, że po standaryzacji zmienne mają zbliżone wariancje, co pozytywnie wpłynie na działanie algorytmów.

Na koniec wstępnej analizy chcemy wskazać cechy o najlepszej zdolności dyskryminacyjnej, czyli takie, które najlepiej rozdzielają nasze odmiany.

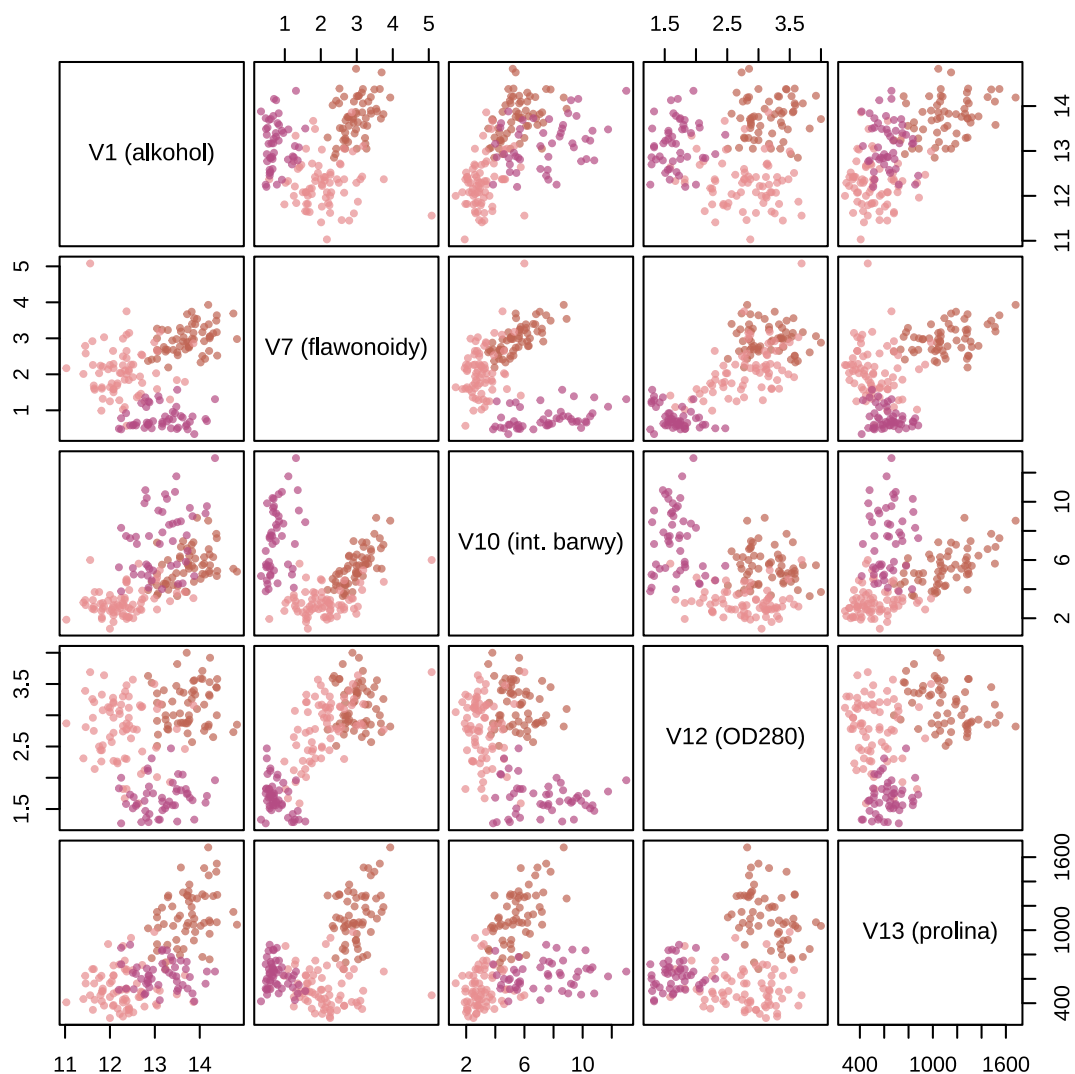
Pomogą nam w tym wykresy pudełkowe poszczególnych cech sporządzone osobno dla każdej odmiany – im mniej pudełka trzech odmian na siebie nachodzą, tym lepiej dana cecha różnicuje wina.



Rysunek 7: Wykresy pudełkowe wszystkich cech w podziale na odmianę; skala osi pionowej dobrana osobno dla każdej cechy. Kolory odpowiadają odmianom: *Barolo*, *Grignolino*, *Barbera*.

Na oko, najmniej nakładające się rozkłady – a więc cechy najlepiej rozdzielające odmiany – to V7 (flawonoidy), V13 (zawartość proliny), V12 (OD280/OD315 rozcieńczonych win), V1 (zawartość alkoholu) oraz V10 (intensywność barwy).

Ten podzbiór cech wykorzystamy dalej, obok pełnego zestawu cech, aby sprawdzić, czy redukcja liczby zmiennych pogarsza jakość klasyfikacji. Ich zdolność do rozdzielania odmian dobrze widać też na wykresie rozrzutu poniżej:



Rysunek 8: Wykres rozrzutu pięciu cech o najlepszej zdolności dyskryminacyjnej. Kolory odmian jak wyżej: *Barolo*, *Grignolino*, *Barbera*.

2.3 Klasyfikacja i ocena dokładności

Będziemy porównywać trzy klasyfikatory: k-najbliższych sąsiadów, drzewa klasyfikacyjne oraz naiwny klasyfikator bayesowski.

Na początku dzielimy zbiór na część uczącą i część testową w proporcjach 2 do 1:

```
set.seed(67)

n <- nrow(dane2) # liczba obserwacji

# losujemy, które obserwacje trafią do zbioru uczącego, a które do testowego
index <- sample(1:n, size = round(2/3 * n))
```

```
train <- dane2[index, ]
test  <- dane2[-index, ]
```

Zbiór uczący zawiera 119 obserwacji, a testowy 59.

2.3.1 Metoda k-najbliższych sąsiadów

Zaczynamy od dokonania klasyfikacji metodą k-najbliższych sąsiadów (wybieramy $k = 5$). Dla lepszego działania algorytmu dokonujemy ponownej standaryzacji na podzbiorze danych.

```
train_x <- scale(train[, -1])
test_x  <- scale(test[, -1],
                  center = attr(train_x, "scaled:center"),
                  scale   = attr(train_x, "scaled:scale"))

train_y <- train[,1]
test_y  <- test[,1]

knn_pred_test <- knn(train_x, test_x, train_y, k = 5)

knn_pred_train <- knn(train_x, train_x, train_y, k = 5)
```

Macierze pomyłek dla zbioru uczącego i zbioru testowego dla k-NN prezentują się następująco:

	<i>Barolo</i>	<i>Grignolino</i>	<i>Barbera</i>
<i>Barolo</i>	38	0	0
<i>Grignolino</i>	1	43	2
<i>Barbera</i>	0	0	35

Tabela 9: Macierz pomyłek – k-NN, zbiór uczący

	<i>Barolo</i>	<i>Grignolino</i>	<i>Barbera</i>
<i>Barolo</i>	21	0	0
<i>Grignolino</i>	2	23	0
<i>Barbera</i>	0	0	13

Tabela 10: Macierz pomyłek – k-NN, zbiór testowy

Widzimy, że algorytm radzi sobie dosyć dobrze. Błąd klasyfikacji dla zbioru uczącego wynosi 2.5% a dla zbioru testowego 3.4%; wartości są relatywnie zbliżone do siebie, co świadczy, że model nie jest przetrenowany.

Nie występuje też zjawisko maskowania danych.

2.3.2 Drzewa klasyfikacyjne

Kolejnym testowanym przez nas algorytmem jest algorytm drzew klasyfikacyjnych (parametr *cp* (*complexity parameter*) – parametr odpowiadający za sterowanie przycinaniem i złożonością drzewa – ustawiony na domyślną wartość 0.01):

```
tree <- rpart(as.factor(train[,1]) ~ ., data = train, method = "class")  
  
tree_pred_test <- predict(tree, test, type = "class")  
  
tree_pred_train <- predict(tree, train, type = "class")
```

Macierze pomyłek dla tego algorytmu wyglądają następująco:

	<i>Barolo</i>	<i>Grignolino</i>	<i>Barbera</i>
<i>Barolo</i>	38	0	0
<i>Grignolino</i>	0	46	0
<i>Barbera</i>	0	0	35

Tabela 11: Macierz pomyłek – drzewo, zbiór uczący

	<i>Barolo</i>	<i>Grignolino</i>	<i>Barbera</i>
<i>Barolo</i>	21	0	0
<i>Grignolino</i>	0	25	0
<i>Barbera</i>	0	0	13

Tabela 12: Macierz pomyłek – drzewo, zbiór testowy

Otrzymane wyniki świadczą o bezbłędnej klasyfikacji zarówno na zbiorze uczącym, jak i testowym, co świadczy o bardzo dobrych własnościach klasyfikacyjnych algorytmu drzew decyzyjnych. Z uwagi na fakt, że podział danych na zbiór uczący i testowy został wykonany poprawnie, a wynik 100% występuje również na danych testowych, możemy wykluczyć przetrenowanie modelu.

2.3.3 Naiwny klasyfikator bayesowski

Następnie, przeprowadzamy klasyfikację naiwnym algorytmem bayesowskim (ang. *naïve Bayes*):

```
nb <- naiveBayes(as.factor(train[,1]) ~ ., data = train)  
  
nb_pred_test <- predict(nb, test)  
  
nb_pred_train <- predict(nb, train)
```

Macierze pomyłek:

	<i>Barolo</i>	<i>Grignolino</i>	<i>Barbera</i>
<i>Barolo</i>	38	0	0
<i>Grignolino</i>	0	46	0
<i>Barbera</i>	0	0	35

Tabela 13: Macierz pomyłek – naïve Bayes, zbiór uczący

	<i>Barolo</i>	<i>Grignolino</i>	<i>Barbera</i>
<i>Barolo</i>	21	0	0
<i>Grignolino</i>	0	25	0
<i>Barbera</i>	0	0	13

Tabela 14: Macierz pomyłek – naïve Bayes, zbiór testowy

Wyniki klasyfikacji dla modelu *naïve Bayes* dla zbioru uczącego wykazują 100% dokładności – model bezbłędnie dopasował dane, natomiast wynik błędu dla zbioru testowego wynosi 0%, co świadczy o bardzo dobrym działaniu modelu.

2.4 Porównanie metod

Błędy klasyfikacji dla testowanych metod wyglądały następująco:

Metoda	Zbiór uczący (%)	Zbiór testowy (%)
k-NN (k=5)	2.5	3.4
drzewo klasyfikacyjne	0.0	0.0
naïve Bayes	0.0	0.0

Tabela 15: Porównanie błędów klasyfikacji testowanych metod (parametry domyślne)

Metoda k-najbliższych sąsiadów skutkuje błędem rzędu 3% na zbiorze testowym, natomiast drzewo i *naïve Bayes* osiągają błąd okrągłego 0%.

Takie dobre wyniki dwóch ostatnich metod mogą nie wynikać jednak z tego że są one 100% skuteczne, lecz ze szczęścia przy losowaniu zbioru uczącego i testowego – ten drugi liczy zaledwie 59 obserwacji, co jest małą próbka.

Z tego powodu, do porównania metod zastosujemy walidację krzyżową (ang. *cross-validation*). Polega ona na podziale zbioru uczącego na k równych części (tzw. *foldów*). Model trenujemy k -krotnie: za każdym razem jeden fold pełni rolę zbioru walidacyjnego, a pozostałe $k - 1$ foldów tworzy zbiór uczący. Błąd klasyfikacji uśredniamy po wszystkich k powtórzeniach,

dzięki czemu każda obserwacja dokładnie raz trafia do zbioru walidacyjnego, a końcowa ocena znacznie słabiej zależy od konkretnego podziału.

Stosujemy 10-krotną walidację krzyżową ($k = 10$) dla wszystkich trzech metod z parametrami domyślnymi:

Metoda	Błąd 10-krotnej CV (%)
k-NN (k=5)	5.2
drzewo klasyfikacyjne	16.8
naïve Bayes	6.2

Tabela 16: Błędy klasyfikacji testowanych metod w 10-krotnej walidacji krzyżowej (parametry domyślne)

Walidacja krzyżowa potwierdza niski błąd metody k-najbliższych sąsiadów (5.2%). W przypadku naiwnego klasyfikatora bayesowskiego błąd rośnie natomiast z 0% do 6.2%, a w przypadku metody drzew klasyfikacyjnych aż do 16.8% – ich bezbłędne wyniki na zbiorze testowym okazują się więc efektem korzystnego losowania, a nie rzeczywistą jakością modelu.

Patrząc na wyniki walidacji krzyżowej, najlepiej wypadają k-NN oraz naïve Bayes, a drzewo klasyfikacyjne jest wyraźnie słabsze.

2.5 Różne parametry oraz podzbiory cech

W następnej sekcji sprawdzamy, jak na te wyniki wpływa dobór parametrów poszczególnych metod oraz ograniczenie liczby cech.

Dla każdej z metod będziemy eksplorować różne wartości parametrów zarówno dla pełnego zestawu cech, jak i dla wybranego podzbioru pięciu wybranych wcześniej najlepiej rozróżniających cech: V7 (flawonoidy), V13 (zawartość proliny), V12 (OD280/OD315 rozcieńczonych win), V1 (zawartość alkoholu) oraz V10 (intensywność barwy).

Wiedząc już, że wyniki dla jednego podziału w dużym stopniu zależą od elementu losowego, będziemy oceniać rozważane warianty używając 10-krotnej walidacji krzyżowej, tak samo jak przy porównywaniu surowych metod (będziemy jednak patrzeć na dokładność zamiast na błąd, czyli na wartość 100% – błąd).

2.5.1 Metoda k-najbliższych sąsiadów

Zaczynamy od metody k-najbliższych sąsiadów. Na pełnym zestawie cech sprawdzamy pięć wartości parametru k : 5, 7, 9, 11 oraz 13. Oczywiście dokonujemy standaryzacji.

k	dokładność
5	0.948
7	0.941
9	0.941
11	0.941
13	0.957

Tabela 17: Wyniki 10-krotnej walidacji krzyżowej – k-NN (wg parametru k)

Najlepszą dokładność (95.7%) uzyskujemy dla $k = 13$, choć różnice między kolejnymi wartościami k są niewielkie – dokładności mieszczą się w przedziale od 94.1% do 95.7% i nie zmieniają się monotonicznie.

To samo powtarzamy na podzbiorze pięciu cech:

k	dokładność
5	0.958
7	0.967
9	0.949
11	0.950
13	0.949

Tabela 18: Wyniki walidacji krzyżowej – k-NN na podzbiorze pięciu cech

Na podzbiorze pięciu cech najwyższą dokładność uzyskano dla $k = 7$, a jej poziom (96.7%) jest podobny do najlepszego wyniku na pełnym zestawie cech (95.7%), a nawet trochę lepszy. Te pięć cech niesie więc niemal całą informację różnicującą odmiany, a redukcja wymiaru praktycznie nie pogarsza klasyfikacji – istnieje nawet możliwość że ją polepsza, choć różnica nie jest wystarczająco duża aby móc to stwierdzić.

2.5.2 Drzewa klasyfikacyjne

W przypadku metody drzewa klasyfikacyjnego, dostosowujemy parametr **cp** (przyp.: *complexity parameter* – parametr odpowiadający za sterowanie przycinaniem i złożonością drzewa).

cp	dokładność
0.0	0.810
0.1	0.809
0.2	0.809
0.3	0.809
0.5	0.387

Tabela 19: Wyniki 10-krotnej walidacji krzyżowej – drzewo (wg parametru cp)

W walidacji krzyżowej najlepszy wynik (81%) uzyskano dla $cp = 0$. Wraz ze wzrostem cp dokładność spada – prawdopodobnie ponieważ zbyt silne przycinanie usuwa informacje.

Tę samą procedurę powtarzamy na podzbiorze pięciu cech:

cp	dokładność
0.0	0.831
0.1	0.781
0.2	0.781
0.3	0.781
0.5	0.387

Tabela 20: Wyniki walidacji krzyżowej – drzewo na podzbiorze pięciu cech

Na podzbiorze pięciu cech dokładność (83.1%) jest zbliżona do wyniku na pełnym zestawie (81%), a nawet odrobinę lepsza, więc również dla drzewa ograniczenie liczby cech nie pogarsza wyników. Dokładność jest lepsza, ale nie na tyle aby móc stwierdzić przewagę ograniczonego zbioru cech.

2.5.3 Naiwny klasyfikator bayesowski

Naiwny klasyfikator bayesowski oceniamy w dwóch wariantach – z estymacją jądrową (`usekernel=TRUE`) i bez niej (`usekernel=FALSE`). Najpierw na pełnym zestawie cech:

usekernel	dokładność
FALSE	0.941
TRUE	0.966

Tabela 21: Wyniki 10-krotnej walidacji krzyżowej – naïve Bayes (wg estymacji jądrowej)

Najlepszy wynik uzyskano dla wariantu z estymacją jądrową, ze średnią dokładnością 96.6%. Świadczy to o bardzo dobrej zgodności przewidywań z rzeczywistymi odmianami oraz o dobrej separowalności klas przy założeniu niezależności cech.

Na koniec powtarzamy ocenę naïve Bayes na podzbiorze pięciu cech:

usekernel	dokładność
FALSE	0.948
TRUE	0.957

Tabela 22: Wyniki walidacji krzyżowej – naïve Bayes na podzbiorze pięciu cech

Również na podzbiorze cech, najlepsza dokładność jest dla `usekernel=TRUE` (95.7%). Pozostaje ona zbliżona do wyniku na pełnym zestawie (96.6%), co potwierdza, że wybrane pięć cech wystarcza do rozróżnienia odmian.

2.6 Podsumowanie

W poniższej tabeli znajduje się podsumowanie dokładności trzech rozważanych metod dla najlepszych parametrów i zestawów cech:

Metoda	Zestaw cech	Najlepszy parametr	Dokładność CV (%)
k-NN	pełny (13 cech)	$k = 13$	95.7
k-NN	podzbiór (5 cech)	$k = 7$	96.7
drzewo	pełny (13 cech)	$cp = 0$	81.0
drzewo	podzbiór (5 cech)	$cp = 0$	83.1
naïve Bayes	pełny (13 cech)	<code>usekernel = TRUE</code>	96.6
naïve Bayes	podzbiór (5 cech)	<code>usekernel = TRUE</code>	95.7

Tabela 23: Najlepsze warianty wszystkich metod (10-krotna walidacja krzyżowa); pogrubiono lepszy wariant w obrębie każdej metody

Oceniamy je na podstawie 10-krotnej walidacji krzyżowej. Ma to duży wpływ na wyniki, gdyż gdybyśmy patrzyli na rezultaty pojedynczej klasyfikacji wyniki byłyby mocno zależne od elementu losowego, a konkretnie podziału na zbiór uczący i testowy. Dzięki użyciu tej metody, wyniki są dokładniejsze.

Metoda k-najbliższych sąsiadów najlepiej sobie radzi w wariancie z $k = 7$ oraz ograniczonym zbiorem cech, z dokładnością 96.7%. Wartość parametru tutaj zbyt wiele nie znaczy, gdyż dla wszystkich testowanych parametrów wychodziły podobne. Wartościowym rezultatem jest jednak niegorszy, a nawet lepszy wynik dla ograniczonego podzbioru cech – wskazuje on na to, że zbiór najlepiej różnicujących cech został dobrze dobrany, przynajmniej dla tego modelu.

Dla algorytmu opartego na drzewach decyzyjnych, najlepsza dokładność (83.1%) wychodzi dla najmniejszej wartości parametru, $cp=0$, oraz ograniczonego zbioru cech. Dla wyższych wartości parametru dokładność była znacząco niższa – atrybutowaliśmy to temu, że zbyt silne przycinanie usuwa informacje. Lepsza dokładność dla ograniczonego zbioru cech jest prawdopodobnie przypadkiem, lecz podobnie jak w przypadku k-NN, dobry wynik wskazuje na wystarczającą ilość informacji zawartą w tym podzbiorze.

Naiwny klasyfikator bayesowski najlepiej sobie radzi dla `usekernel=TRUE` oraz pełnego zbioru danych – dokładność wynosi 96.6%. Taka wartość parametru dawała lepsze wyniki przy obu zestawach cech, więc najprawdopodobniej jest faktycznie lepsza. W przeciwieństwie do dwóch poprzednich metod, ograniczony zbiór danych daje odrobinę niższą dokładność, jednak spadek jest niski, więc także świadczy to o tym, że najwięcej informacji jest zawarta w zmiennych z podzbioru.

Fakt, że z 13 cech, wybrany podzbiór pięciu daje dobre wyniki klasyfikacji, może sugerować że te cechy faktycznie najlepiej rozróżniają rozważane odmiany win – nasz podzbiór cech składał się ze zmiennych zawierających informacje o zawartości flawonoidów, proliny oraz alkoholu, OD280/OD315 rozcieńczonych win oraz intensywności barwy, więc bardzo możliwe jest że to właśnie tymi cechami różnią się te odmiany.

Najlepszymi metodami do klasyfikacji wydają się być metoda k-najbliższych sąsiadów oraz naiwnego klasyfikatora bayesowskiego. Obie poprawnie klasyfikują przynależność wina do odmiany w około 96 – 97% przypadków. Algorytm drzew decyzyjnych wypada przy nich zauważalnie gorzej, więc może nie być najlepszym wyborem dla degustatorów chcących efektywnie sklasyfikować odmianę wina.